

Relatório Trabalho Prático 1

Comando Vigia Pirata

Sistemas Operativos

Estudantes: Adriana Martins [132842] e Alice Tembe [134713]

2026/2027

Índice

1. Introdução..... 2
1.1 Objetivos do Trabalho..... 2
1.2 Modos e Deteções..... 2
1.3 Arquitetura / abordagem..... 3
2. Solução..... 3
2.1 Divisão de Tarefas..... 3
2.2 Formato da Base de Dados..... 3
2.3 Fluxo do -p..... 4
2.4 Fluxo do -v..... 4
3. Criação da Base de Dados..... 4
3.1 Validação de Argumentos da Base de Dados..... 5
3.2. Travessia de Diretórios e Extração de Metadados..... 6
4. Interface e Verificação..... 6
4.1 Leitura das opções..... 6
4.2 Validações de erro e uso()..... 8
4.3 Leitura da BD e comparações..... 8
4.4 Ficheiros novos e o array associativo registados..... 9
4.5 O -e na verificação..... 10
4.6 O -s com logger..... 11
4.7 rm -f e chmod 444..... 11
4.8 Dificuldades e decisões..... 11
5. Testes..... 12
5.1 Opção Inválida: ./vp.sh -x..... 12
5.2 -f sem valor: ./vp.sh -f echo $?..... 12
5.3 -e sem valor: ./vp -p -v echo $?..... 12
5.4 -p e -v juntos: ./vp.sh -p -v echo $?..... 13
5.5 -p sem diretório: ./vp.sh -p..... 13
5.6 Diretório que não existe: ./vp.sh -p /nao/existe -f bd_x..... 13
5.7 Base de dados em falta: ./vp.sh -v -f nao_existe..... 13
5.8 Sem opções (modo e BD por defeito): ./vp.sh..... 13
5.9 Criar a BD: ./vp.sh -p projeto -f bd_final..... 14
5.10 Verificar as permissões da BD: ls -l bd_final..... 14
5.11 Verificar o conteúdo da BD: cat bd_final..... 14
5.12 Tentar escrever na BD: echo x >> bd_final..... 14
5.13 Proteger várias pastas: ./vp.sh -p projeto outra -f bd2 -s..... 14
5.14 Recriar uma BD existente: ./vp.sh -p projeto -f bd..... 14
5.15 Verificação sem alterações: ./vp.sh -v -f bd..... 14
5.16 Modo de verificação por defeito: ./vp.sh -f bd..... 14
5.17 Alteração apenas do conteúdo: ./vp.sh -v -f bd..... 14
5.18 Alteração do conteúdo e tamanho: ./vp.sh -v -f bd..... 14
5.19 Alteração das permissões: ./vp.sh -v -f bd..... 15

5.20 Alteração apenas da data: ./vp.sh -v -f bd.....15
5.21 Alteração da posse: ./vp.sh -v -f bd.....15
5.22 Deteção de ficheiro apagado: ./vp.sh -v -f bd.....15
5.23 Deteção de ficheiro novo: ./vp.sh -v -f bd.....15
5.24 Deteção simultânea de várias alterações: ./vp.sh -v -f bd.....15
5.25 Exclusão de ficheiros .log: ./vp.sh -p projeto -f bd_e -e '*.log'.....15
5.26 Verificação do conteúdo da BD com exclusões: cat bd_e..... 15
5.27 Exclusão durante a verificação: ./vp.sh -v -f bd_e.....16
5.28 Registo de alertas com ./vp.sh -v -f bd -s.....16
5.29 Verificação sem registo de alertas: ./vp.sh -v -f bd.....16
5.30 Deteção de ficheiros em subdiretórios: ./vp.sh -v -f bd..... 16
5.31 Tratamento de nomes com espaços: ./vp.sh -p 'com espacos' -f bd_sp.....16
5.32 Base de dados dentro do diretório protegido: ./vp.sh -p projeto -f projeto/bd2..... 16
5.33 Utilização da BD por defeito: ./vp.sh -p projeto.....16
5.34 Verificação da sintaxe: bash -n vp.sh.....17
5.35 Verificação da versão do Bash: bash --version | head -1.....17
6. Limitações..... 17
7. Conclusão..... 18
8. Bibliografia..... 18

1. Introdução

O presente trabalho consiste na criação de um shell script em Bash para monitorização e deteção de intrusões, denominado VP (Vigia-Pirata). A segurança e a verificação da integridade dos dados são pilares da administração de sistemas operativos: uma intrusão deixa muitas vezes rasto sob a forma de ficheiros criados, apagados ou alterados, e detetá-lo cedo limita os danos.

1.1 Objetivos do Trabalho

O trabalho foi desenvolvido com dois propósitos. O primeiro é criar um mecanismo de proteção que mapeie o estado inicial dos diretórios críticos definidos pelo utilizador, preservando os metadados e o SHA-256 de cada ficheiro numa base de dados (BD) de referência. O segundo é implementar uma verificação que compare o estado atual com essa BD e identifique claramente a criação, a modificação ou a remoção de ficheiros.

1.2 Modos e Deteções

O vp funciona a partir de dois modos que não podem ser utilizados em simultâneo: Proteção (-p <dir1> [<dir2>...]), este percorre os diretórios e subdiretórios, regista os metadados e o SHA-256 de cada ficheiro e guarda tudo numa base de dados. E o modo Verificação, -v, este sendo modo por defeito, compara o estado atual com a base de dados e reporta as diferenças.

Além disso, existem ainda duas opções auxiliares, -f <ficheiro> e -e <padrão>, a primeira escolhe o ficheiro da base de dados, por defeito /tmp/.pdb e a segunda opção exclui ficheiros por padrão e -s envia os alertas para o sistema de logging.

Em Verificação, o vp deteta as seguintes situações:

- Ficheiro Novo, reportada como: NOVO
- Ficheiro Apagado, reportada como: APAGADO
- Conteúdo Alterado (SHA-256), reportada como: MODIFICADO (... Conteúdo)
- Tamanho Alterado, reportada como: MODIFICADO (... Tamanho)
- Permissões Alteradas, reportada como: MODIFICADO (... Permissões)
- Data de Modificação Alterada, reportada como: MODIFICADO (... Data)

- Posse Alterada (dono ou grupo), reportada como: MODIFICADO (... Posse)

1.3 Arquitetura / abordagem

O desenvolvimento seguiu uma estratégia incremental, assente na separação de preocupações e no princípio da responsabilidade única. A recolha dos dados de referência (Proteção) está separada da lógica de análise e comparação (Verificação), o que facilita a manutenção e a leitura do código. O resultado final é um único ficheiro, vp_p2.sh (abreviado para vp neste relatório), com duas partes: a função gerar_base_dados no início e, a seguir, a interface e a verificação.

2. Solução

2.1 Divisão de Tarefas

A Alice Tembe ficou com a criação da Base de Dados; a Adriana Martins ficou com a Interface, a Verificação e o README. Foram decididos em conjunto o formato da BD, a forma de separar os campos, a ordem de leitura dos ficheiros e a entrega do trabalho. O relatório e a apresentação foram feitos pelas duas.

2.2 Formato da Base de Dados

A Base de Dados é um ficheiro de texto com três tipos de linha:
#DIR /tmp/amb , uma linha por diretório protegido
#EXCLUIR *.log , o padrão do -e
/tmp/amb/f1.txt|utilizador|grupo|2|664|1792446831|73cb3858… , uma linha por ficheiro, com sete campos separados por “|”: caminho | dono | grupo | tamanho | permissões (octal) | data de modificação (segundos desde 1970) | SHA-256 .
Ao guardar os diretórios e o padrão na Base de Dados, permite que a verificação saiba o que voltar a percorrer sem o utilizador repetir -p ou -e. As linhas que comecem por # são metadados, e é por isso que o número de ficheiros processados se obtém com: grep -vc ‘^#’ .

2.3 Fluxo do -p

1. Ler as opções e guardar os diretórios.
2. Validar (modos em conflito, diretórios em falta, diretórios que não existem).
3. Apagar a BD antiga (rm -f).
4. Chamar gerar_base_dados (Alice), que escreve #DIR, #EXCLUIR e uma linha por ficheiro.
5. Proteger a BD contra escrita (chmod 444) e mostrar o resumo (ficheiros processados e diretórios protegidos).

2.4 Fluxo do -v

1. Ler as opções e confirmar que a BD existe.
2. Percorrer as linhas de ficheiros da BD: se o ficheiro já não existe é APAGADO; se existe, comparam-se os cinco campos e, havendo diferenças, é MODIFICADO.
3. Reler da BD os diretórios (#DIR) e o padrão (#EXCLUIR).
4. Percorrer de novo esses diretórios e reportar como NOVO o que não estava na BD.
5. Mostrar o resumo: N verificados | N Modificados | N Apagados | N Novos.

3. Criação da Base de Dados

A implementação da função gerar_base_dados, é responsável por construir o ficheiro de base de dados com o registo de metadados e hashes dos ficheiros monitorizados.

```bash
gerar_base_dados() {
    local ficheiro_bd="$1"
    local padrao_exclusao="$2"

    shift 2

    [ $# -eq 0 ] && return 1

    > "$ficheiro_bd"

    # Guarda os diretórios e o padrão de exclusão nos metadados da BD
    for dir in "$@"; do
        echo "#DIR $dir" >> "$ficheiro_bd"
    done

    echo "#EXCLUIR $padrao_exclusao" >> "$ficheiro_bd"

    # Percorre os diretórios e regista os dados de cada ficheiro encontrado
    for dir in "$@"; do

        [[ ! -d "$dir" ]] && continue

        while IFS= read -r -d '' ficheiro; do

            [[ ! -f "$ficheiro" ]] && continue
            [[ "$ficheiro" -ef "$ficheiro_bd" ]] && continue

            if [[ -n "$padrao_exclusao" ]]; then
                [[ "$ficheiro" == $padrao_exclusao ]] && continue
            fi

            local caminho="$ficheiro"
            local user=$(stat -c '%U' -- "$ficheiro" 2>/dev/null)
            local grupo=$(stat -c '%G' -- "$ficheiro" 2>/dev/null)
            local tamanho=$(stat -c '%s' -- "$ficheiro" 2>/dev/null)
            local permissoes=$(stat -c '%a' -- "$ficheiro" 2>/dev/null)
            local data=$(stat -c '%Y' -- "$ficheiro" 2>/dev/null)
            local sha256=$(sha256sum -- "$ficheiro" 2>/dev/null | awk '{print $1}')

            echo "$caminho|$user|$grupo|$tamanho|$permissoes|$data|$sha256" >> "$ficheiro_bd"

        done < <(find "$dir" -type f -print0)

    done
}
```

3.1 Validação de Argumentos da Base de Dados

A implementação da função gerar_base_dados inicia-se com a recolha e validação rigorosa dos argumentos fornecidos por linha de comandos. Em primeiro lugar, a função armazena o caminho do ficheiro de destino na variável ficheiro_bd ($1) e o padrão de exclusão de ficheiros em padrao_exclusao ($2). De seguida, é aplicado o comando shift 2 para remover estes dois primeiros parâmetros da lista de argumentos, isolando assim a lista de diretórios a analisar contida em $@. A validação é concluída com a instrução [$# -eq 0], que verifica se restou pelo menos um diretório válido para processamento. Por fim, procede-se à reinicialização do

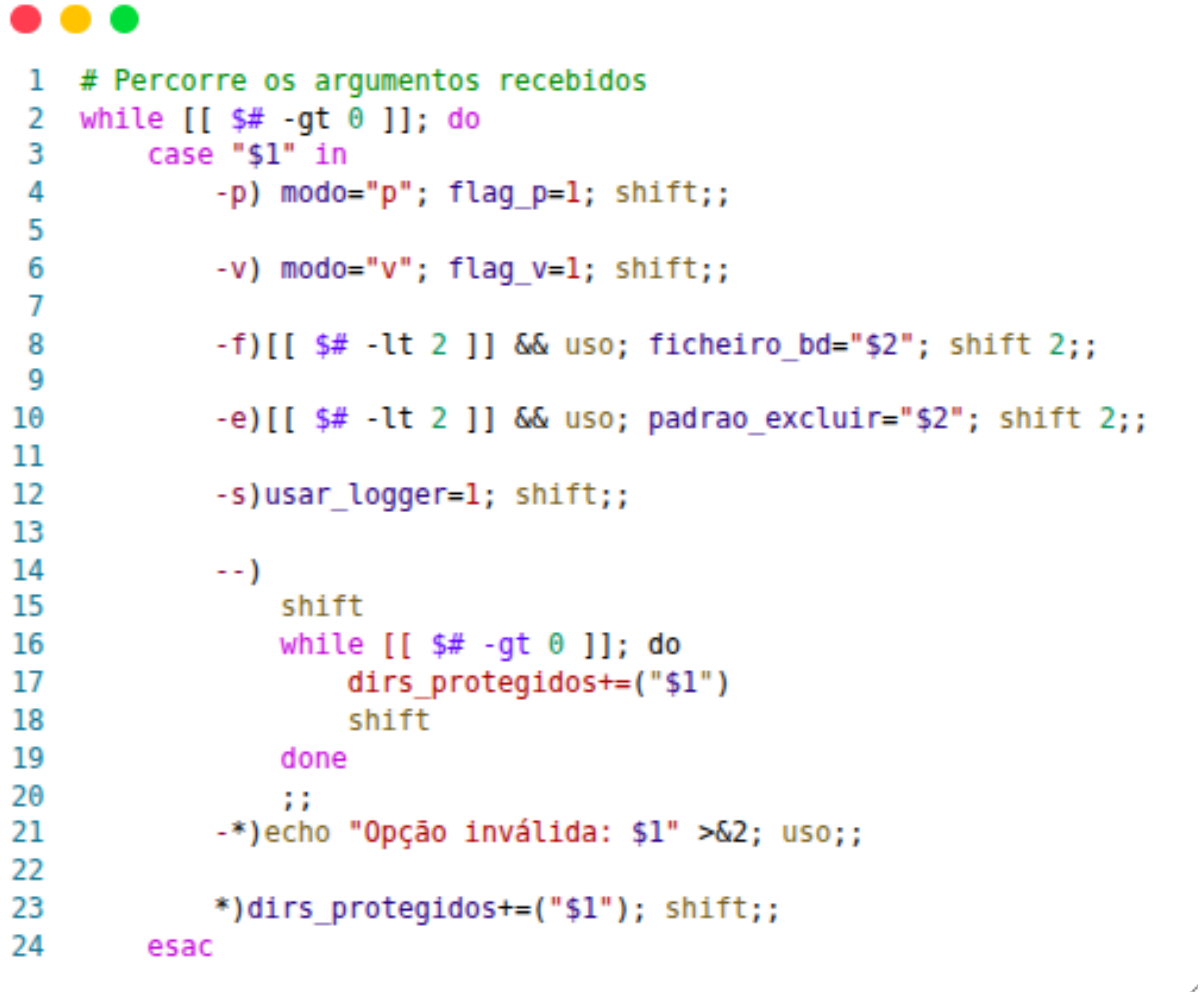
ficheiro de destino através do operador > "$ficheiro_bd", onde são gravadas as linhas de cabeçalho iniciais com os diretórios protegidos (#DIRS) e os padrões de exclusão (#EXCLUIR).

3.2. Travessia de Diretórios e Extração de Metadados

Após a fase de inicialização, a função passa à varredura e extração de informação da árvore de ficheiros. O script percorre cada diretório da lista contida em "$@", ignorando alvos inválidos com a condição [[! -d "$dir"]]. Para garantir uma travessia segura e robusta, utiliza-se a estrutura de leitura por linhas combinada com o comando find (find "$dir" -type f), o que permite iterar sobre cada ficheiro individualmente sem problemas causados por espaços em branco nos caminhos. Durante a leitura, o script valida e descarta qualquer ficheiro que corresponda ao padrão de exclusão definido ([["$ficheiro" == $padrao_exclusao]]). Para cada ficheiro válido, são recolhidos os respetivos metadados através do comando stat — nomeadamente o utilizador (%U), grupo (%G), tamanho (%s), permissões (%a) e data de modificação (%Y) , os quais são combinados com a respetiva assinatura digital gerada por sha256sum. Toda esta informação recolhida é formatada e gravada sequencialmente no ficheiro da base de dados, utilizando o carater | como delimitador de campos

4. Interface e Verificação

4.1 Leitura das opções



```
# Percorre os argumentos recebidos
while [[ $# -gt 0 ]]; do
    case "$1" in
        -p) modo="p"; flag_p=1; shift;;

        -v) modo="v"; flag_v=1; shift;;

        -f)[[ $# -lt 2 ]] && uso; ficheiro_bd="$2"; shift 2;;

        -e)[[ $# -lt 2 ]] && uso; padrao_excluir="$2"; shift 2;;

        -s)usar_logger=1; shift;;

        --)
            shift
            while [[ $# -gt 0 ]]; do
                dirs_protegidos+=("$1")
                shift
            done
            ;;
        -*)echo "Opção inválida: $1" >&2; uso;;

        *)dirs_protegidos+=("$1"); shift;;
    esac
```

Cada volta olha para $1; as opções simples consomem um argumento (shift) e as que levam valor (-f, -e) consomem dois (shift 2), depois de confirmar que o valor existe. O -- marca o fim das opções: tudo o que vem depois é diretório, o que permite proteger um diretório cujo nome começa por -. Qualquer outra palavra que comece por - é opção inválida, e o resto são diretórios. Com este ciclo as opções e os diretórios podem vir por qualquer ordem.

Porque um array. Os diretórios ficam em dirs_protegidos=() e passam-se com "${dirs_protegidos[@]}", de modo que cada diretório é um elemento e um nome com espaços não se parte em dois. Na primeira versão a lista era uma string ("$dirs $1"), e isso obrigava a expandi-la sem aspas, partindo esses nomes.

4.2 Validações de erro e uso()

Por esta ordem, o programa verifica: (1) -p e -v não podem vir juntos; (2) o -p precisa de pelo menos um diretório; (3) sem -p nem -v, o modo é verificação; (4) na verificação, a BD tem de existir; (5) no -p, todos os diretórios têm de existir, e isto é feito antes de apagar a BD anterior, para uma falha não destruir uma BD boa.

Cada erro escreve a mensagem para o erro padrão (>&2) e chama uso(), que mostra a sintaxe e as opções e termina com exit 1. Quando -f ou -e aparecem sem valor, não há mensagem própria: o vp chama logo uso(). O texto de uso() sai no stdout; só as mensagens de erro vão para o stderr.

4.3 Leitura da BD e comparações

A BD é lida linha a linha. As linhas #... são ignoradas e as restantes partem-se nos sete campos com IFS='|' read -r caminho user grupo tamanho permissoes data sha256 <<< "$linha". Se o ficheiro já não existe, é APAGADO. Se existe, volta-se a recolher cada metadado e comparam-se, um if por campo, juntando o nome do que mudou à variável alteracoes:

```
    alteracoes=""

    # Compara o conteúdo do ficheiro
    if [[ "$sha256_atual" != "$sha256" ]]; then
        alteracoes="$alteracoes Conteúdo"
    fi

    # Compara o tamanho atual com o tamanho inicial
    if [[ "$tamanho" != "$tamanho_atual" ]]; then
        alteracoes="$alteracoes Tamanho"
    fi

    # Verifica se as permissões do ficheiro foram alteradas
    if [[ "$permissoes" != "$perm_atual" ]]; then
        alteracoes="$alteracoes Permissões"
    fi

    # Compara as datas de modificação do ficheiro
    if [[ "$data" != "$data_atual" ]]; then
        alteracoes="$alteracoes Data"
    fi

    # Verifica se o utilizador ou o grupo proprietário foram alterados
    if [[ "$user" != "$user_atual" || "$grupo" != "$grupo_atual" ]]; then
        alteracoes="$alteracoes Posse"
    fi

    # Apresenta os ficheiros modificados e as respetivas diferenças
    if [[ -n "$alteracoes" ]]; then

        echo "$caminho foi MODIFICADO (Alteração de:$alteracoes)"
        alerta "$caminho foi MODIFICADO (Alteração de:$alteracoes)"

        modificados=$((modificados + 1))

        echo "  Informação inicial: USER=$user, GROUP=$grupo, SIZE=$tamanho, PERM=$permissoes, SHA256=$sha256"
        echo "  Informação atual: USER=$user_atual, GROUP=$grupo_atual, SIZE=$tamanho_atual, PERM=$perm_atual, SHA256=$sha256_atual"

    fi

fi
```

Se alteracoes não estiver vazia, escreve-se uma só mensagem por ficheiro, por exemplo MODIFICADO (Alteração de: Conteúdo Tamanho Data), seguida das linhas «Informação inicial» (da BD) e «Informação atual» (lida agora), e o contador de modificados sobe 1 por ficheiro, não por campo. Os apagados mostram a informação inicial e, em vez da atual, a indicação de que o ficheiro não foi encontrado. Nas linhas de informação aparecem USER, GROUP, SIZE, PERM e SHA256 mas não a data; numa alteração só de data as duas linhas são iguais e é a mensagem principal (... Data) que a identifica.

4.4 Ficheiros novos e o array associativo registados

Para saber o que é novo é preciso saber o que já estava na BD. Enquanto se lê a BD, cada caminho é guardado num array associativo, registados["$caminho"]=1. Depois relêem-se da BD os diretórios e o padrão (case sobre "#DIR "* e "#EXCLUIR "*), percorrem-se os diretórios com find -print0 e testa-se:

```bash
# Verifica se o ficheiro já estava registado na base de dados
if [[ -z ${registados["$ficheiro"]+x} ]]; then

    echo "$ficheiro é NOVO"
    alerta "$ficheiro é NOVO"

    # Recolhe os dados atuais dos ficheiros novos encontrados
    sha256_atual=$(sha256sum -- "$ficheiro" | awk '{print $1}')
    user_atual=$(stat -c %U -- "$ficheiro")
    grupo_atual=$(stat -c %G -- "$ficheiro")
    tamanho_atual=$(stat -c %s -- "$ficheiro")
    perm_atual=$(stat -c %a -- "$ficheiro")

    echo "  Informação atual: USER=$user_atual, GROUP=$grupo_atual, SIZE=$tamanho_atual, PERM=$perm_atual, SHA256=$sha256_atual"

    novos=$((novos + 1))

    # Regista o caminho para evitar contagens duplicadas
    registados["$ficheiro"]=1

fi
```

O ${registados["$ficheiro"]+x} só expande para x se a chave existir; se vier vazio, o ficheiro não estava registado. Depois de contado, o ficheiro é registado para não ser reportado duas vezes como novo (por exemplo, se dois diretórios protegidos se sobrepuserem). Esta proteção cobre só os novos: os registos duplicados que a Proteção pode criar nesse caso estão descritos na secção 6.

Porquê um array. A primeira versão procurava cada ficheiro com grep -q -F "$ficheiro$|""ficheiro_bd", o que lança um processo por ficheiro e podia dar falsos positivos, porque o grep encontra o texto em qualquer ponto da linha. O array dá uma consulta direta, sem reler a BD.

4.5 O -e na verificação

O padrão fica guardado na BD (#EXCLUIR) e é relido na verificação, por isso o mesmo critério aplica-se nos dois modos sem o utilizador o repetir, e os ficheiros excluídos não aparecem como NOVO. Um -e passado diretamente ao -v é lido mas não é usado: vale o padrão da BD (teste 13). O padrão é comparado com o caminho completo, e como o * também corresponde a /, *.log apanha ficheiros .log em qualquer subdiretório. O padrão deve ir entre aspas para o shell não o expandir antes do vp.

4.6 O -s com logger

A função alerta só faz algo se -s estiver ativo:

```
# Envia alertas para o sistema de logging quando a opção -s está ativa
alerta() {
    if [[ $usar_logger == 1 ]]; then
        logger -t vp -p user.warning "$1"
    fi
}
```

É chamada junto ao echo de cada APAGADO, MODIFICADO e NOVO, de forma que o ecrã e o log têm a mesma mensagem principal (as linhas de «Informação» só aparecem no ecrã). O -t vp marca as entradas com a etiqueta vp e o -p user.warning dá-lhes prioridade de aviso; consultam-se com journalctl -t vp. Os alertas só existem em verificação; o modo -p não os emite.

4.7 rm -f e chmod 444

Depois de criada, a BD é posta só de leitura (chmod 444) para dificultar que seja alterada por engano ou por um utilizador sem privilégios. Isto tem uma consequência: um -p seguinte não consegue reescrever o ficheiro (o > "$ficheiro_bd" falharia com «Permission denied»). Por isso o programa faz rm -f "$ficheiro_bd" antes de chamar gerar_base_dados: apagar só depende das permissões do diretório, e o -f evita a pergunta de confirmação para ficheiros protegidos (teste 16).

4.8 Dificuldades e decisões

- **Opções depois dos diretórios.** Com getopts, ./vp_p2.sh -p /tmp/amb -f bd dava «o -p precisa de pelo menos um diretório», porque o getopts pára no primeiro argumento que não é opção. Passámos para o ciclo while + case.
- **Lista de diretórios.** De string para array, para suportar nomes com espaços.
- **Ficheiros novos.** De grep por ficheiro para o array associativo registados.
- **Várias alterações, uma mensagem.** Em vez de uma mensagem por campo, juntar tudo em alteracoes.
- **Informação atual** também em apagados e em novos, para o utilizador ver sempre «antes» e «depois» quando existem.
- **Um só ficheiro.** A primeira integração carregava a parte da Alice com source ./parte_a.sh, o que obrigava a entregar dois ficheiros na mesma pasta; na entrega a função ficou no início do mesmo ficheiro.
-

5. Testes

5.1 Opção Inválida: ./vp.sh -x

```
Opção inválida: -x
Uso: ./vp [-p <dir1> [<dir2>...]] [-v] [-f <ficheiro>] [-e <padrao>] [-s]
    -p : Protege um ou mais diretórios
    -v : Verifica alterações nos ficheiros protegidos
    -f : Indica o ficheiro onde a base de dados será guardada
    -e : Exclui ficheiros que correspondam ao padrão indicado
    -s : Envia alertas para o sistema de logging usando logger
```

5.2 -f sem valor: ./vp.sh -f echo $?

```
Uso: ./vp [-p <dir1> [<dir2>...]] [-v] [-f <ficheiro>] [-e <padrao>] [-s]
    -p : Protege um ou mais diretórios
    -v : Verifica alterações nos ficheiros protegidos
    -f : Indica o ficheiro onde a base de dados será guardada
    -e : Exclui ficheiros que correspondam ao padrão indicado
    -s : Envia alertas para o sistema de logging usando logger
1
```

5.3 -e sem valor: ./vp -p -v echo $?

```
Erro: não podes usar -p e -v ao mesmo tempo
Uso: ./vp [-p <dir1> [<dir2>...]] [-v] [-f <ficheiro>] [-e <padrao>] [-s]
    -p : Protege um ou mais diretórios
    -v : Verifica alterações nos ficheiros protegidos
    -f : Indica o ficheiro onde a base de dados será guardada
    -e : Exclui ficheiros que correspondam ao padrão indicado
    -s : Envia alertas para o sistema de logging usando logger
1
```

5.4 -p e -v juntos: ./vp.sh -p -v echo $?

```
Erro: não podes usar -p e -v ao mesmo tempo
Uso: ./vp [-p <dir1> [<dir2>...]] [-v] [-f <ficheiro>] [-e <padrao>] [-s]
    -p : Protege um ou mais diretórios
    -v : Verifica alterações nos ficheiros protegidos
    -f : Indica o ficheiro onde a base de dados será guardada
    -e : Exclui ficheiros que correspondam ao padrão indicado
    -s : Envia alertas para o sistema de logging usando logger
1
```

5.5 -p sem diretório: ./vp.sh -p

```
Erro: o -p precisa de pelo menos um diretório
Uso: ./vp [-p <dir1> [<dir2>...]] [-v] [-f <ficheiro>] [-e <padrao>] [-s]
    -p : Protege um ou mais diretórios
    -v : Verifica alterações nos ficheiros protegidos
    -f : Indica o ficheiro onde a base de dados será guardada
    -e : Exclui ficheiros que correspondam ao padrão indicado
    -s : Envia alertas para o sistema de logging usando logger
```

5.6 Diretório que não existe: ./vp.sh -p /nao/existe -f bd_x

```
Erro: '/nao/existe' não é um diretório
Uso: ./vp [-p <dir1> [<dir2>...]] [-v] [-f <ficheiro>] [-e <padrao>] [-s]
    -p : Protege um ou mais diretórios
    -v : Verifica alterações nos ficheiros protegidos
    -f : Indica o ficheiro onde a base de dados será guardada
    -e : Exclui ficheiros que correspondam ao padrão indicado
    -s : Envia alertas para o sistema de logging usando logger
```

5.7 Base de dados em falta: ./vp.sh -v -f nao_existe

```
Erro: base de dados 'nao_existe' não encontrada
Uso: ./vp [-p <dir1> [<dir2>...]] [-v] [-f <ficheiro>] [-e <padrao>] [-s]
    -p : Protege um ou mais diretórios
    -v : Verifica alterações nos ficheiros protegidos
    -f : Indica o ficheiro onde a base de dados será guardada
    -e : Exclui ficheiros que correspondam ao padrão indicado
    -s : Envia alertas para o sistema de logging usando logger
```

5.8 Sem opções (modo e BD por defeito): ./vp.sh

```
Erro: base de dados '/tmp/.pdb' não encontrada
Uso: ./vp [-p <dir1> [<dir2>...]] [-v] [-f <ficheiro>] [-e <padrao>] [-s]
    -p : Protege um ou mais diretórios
    -v : Verifica alterações nos ficheiros protegidos
    -f : Indica o ficheiro onde a base de dados será guardada
    -e : Exclui ficheiros que correspondam ao padrão indicado
    -s : Envia alertas para o sistema de logging usando logger
```

5.9 Criar a BD: ./vp.sh -p projeto -f bd_final

```
Base de dados de proteção bd_final foi criada com sucesso.
Ficheiros processados: 3
Diretórios protegidos: projeto
```

5.10 Verificar as permissões da BD: ls -l bd_final

```
-r--r--r-- 1 adrianamoranguita adrianamoranguita 429 Nov  5 01:57 bd_final
```

5.11 Verificar o conteúdo da BD: cat bd_final

```
#DIR projeto
#EXCLUIR
projeto/notas.txt|adrianamoranguita|adrianamoranguita|2|664|1793843804|87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
projeto/app.log|adrianamoranguita|adrianamoranguita|2|664|1793843804|6d7ebc44c5bc26207e62f4f628f912e1a0f41ed11764891aa7dd99eab83228e7
projeto/dados.txt|adrianamoranguita|adrianamoranguita|2|664|1793843804|0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
```

5.12 Tentar escrever na BD: echo x >> bd_final

```
bash: bd_final: Permission denied
```

5.13 Proteger várias pastas: ./vp.sh -p projeto outra -f bd2 -s

```
Base de dados de proteção bd2 foi criada com sucesso.
Ficheiros processados: 4
Diretórios protegidos: projeto outra
```

5.14 Recriar uma BD existente: ./vp.sh -p projeto -f bd

```
Base de dados de proteção bd foi criada com sucesso.
Ficheiros processados: 3
Diretórios protegidos: projeto
```

5.15 Verificação sem alterações: ./vp.sh -v -f bd

```
Resumo da Verificação: 3 verificados | 0 Modificados | 0 Apagados | 0 Novos
```

5.16 Modo de verificação por defeito: ./vp.sh -f bd

```
Resumo da Verificação: 3 verificados | 0 Modificados | 0 Apagados | 0 Novos
```

5.17 Alteração apenas do conteúdo: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Data)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=c865f6c5ab8d1b0bcd383a5e1e3879d22681c96bf462c269b7581d523fbe70ab
Resumo da Verificação: 3 verificados | 1 Modificados | 0 Apagados | 0 Novos
```

5.18 Alteração do conteúdo e tamanho: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=7, PERM=664, SHA256=fdfd58ed59e0bc9368a0200c4de4c81eaa51ad5955c58df7a82bf10b156ff5db
Resumo da Verificação: 3 verificados | 1 Modificados | 0 Apagados | 0 Novos
```

5.19 Alteração das permissões: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=7, PERM=664, SHA256=fdfd58ed59e0bc9368a0200c4de4c81eaa51ad5955c58df7a82bf10b156ff5db
projeto/dados.txt foi MODIFICADO (Alteração de: Permissões)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=600, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
Resumo da Verificação: 3 verificados | 2 Modificados | 0 Apagados | 0 Novos
```

5.20 Alteração apenas da data: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=7, PERM=664, SHA256=fdfd58ed59e0bc9368a0200c4de4c81eaa51ad5955c58df7a82bf10b156ff5db
projeto/dados.txt foi MODIFICADO (Alteração de: Permissões)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=600, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
Resumo da Verificação: 3 verificados | 2 Modificados | 0 Apagados | 0 Novos
```

5.21 Alteração da posse: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data Posse)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=root, GROUP=adrianamoranguita, SIZE=7, PERM=664, SHA256=fdfd58ed59e0bc9368a0200c4de4c81eaa51ad5955c58df7a82bf10b156ff5db
projeto/dados.txt foi MODIFICADO (Alteração de: Permissões)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=600, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
Resumo da Verificação: 3 verificados | 2 Modificados | 0 Apagados | 0 Novos
```

5.22 Deteção de ficheiro apagado: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data Posse)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=root, GROUP=adrianamoranguita, SIZE=7, PERM=664, SHA256=fdfd58ed59e0bc9368a0200c4de4c81eaa51ad5955c58df7a82bf10b156ff5db
projeto/dados.txt foi APAGADO
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: [Ficheiro não encontrado no sistema]
Resumo da Verificação: 3 verificados | 1 Modificados | 1 Apagados | 0 Novos
```

5.23 Deteção de ficheiro novo: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data Posse)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=root, GROUP=adrianamoranguita, SIZE=7, PERM=664, SHA256=fdfd58ed59e0bc9368a0200c4de4c81eaa51ad5955c58df7a82bf10b156ff5db
projeto/dados.txt foi APAGADO
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: [Ficheiro não encontrado no sistema]
projeto/novo.txt é NOVO
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=a4fb621495a0122493b2203591c448903c472e306a1ede54fabad829e01075c0
Resumo da Verificação: 3 verificados | 1 Modificados | 1 Apagados | 1 Novos
```

5.24 Deteção simultânea de várias alterações: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data Posse)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=root, GROUP=adrianamoranguita, SIZE=12, PERM=664, SHA256=2a7384e2d430093d1099cec026e31336d13b32a043851c06e606b74e7b89446e
projeto/dados.txt foi APAGADO
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: [Ficheiro não encontrado no sistema]
projeto/novo.txt é NOVO
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=a4fb621495a0122493b2203591c448903c472e306a1ede54fabad829e01075c0
Resumo da Verificação: 3 verificados | 1 Modificados | 1 Apagados | 1 Novos
```

5.25 Exclusão de ficheiros .log: ./vp.sh -p projeto -f bd_e -e '*.log'

```
Base de dados de proteção bd_e foi criada com sucesso.
Ficheiros processados: 2
Diretórios protegidos: projeto
```

5.26 Verificação do conteúdo da BD com exclusões: cat bd_e

```
#DIR projeto
#EXCLUIR *.log
projeto/notas.txt|root|adrianamoranguita|12|664|1793845480|2a7384e2d430093d1099cec026e31336d13b32a043851c06e606b74e7b89446e
projeto/novo.txt|adrianamoranguita|adrianamoranguita|2|664|1793845480|a4fb621495a0122493b2203591c448903c472e306a1ede54fabad829e01075c0
```

5.27 Exclusão durante a verificação: ./vp.sh -v -f bd_e

```
Resumo da Verificação: 2 verificados | 0 Modificados | 0 Apagados | 0 Novos
```

5.28 Registo de alertas com ./vp.sh -v -f bd -s

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data Posse)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=root, GROUP=adrianamoranguita, SIZE=12, PERM=664, SHA256=2a7384e2d430093d1099cec026e31336d13b32a043851c06e606b74e7b89446e
projeto/dados.txt foi APAGADO
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: [Ficheiro não encontrado no sistema]
projeto/novo.txt é NOVO
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=a4fb621495a0122493b2203591c448903c472e306a1ede54fabad829e01075c0
Resumo da Verificação: 3 verificados | 1 Modificados | 1 Apagados | 1 Novos
```

5.29 Verificação sem registo de alertas: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data Posse)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=root, GROUP=adrianamoranguita, SIZE=12, PERM=664, SHA256=2a7384e2d430093d1099cec026e31336d13b32a043851c06e606b74e7b89446e
projeto/dados.txt foi APAGADO
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: [Ficheiro não encontrado no sistema]
projeto/novo.txt é NOVO
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=a4fb621495a0122493b2203591c448903c472e306a1ede54fabad829e01075c0
Resumo da Verificação: 3 verificados | 1 Modificados | 1 Apagados | 1 Novos
```

5.30 Deteção de ficheiros em subdiretórios: ./vp.sh -v -f bd

```
projeto/notas.txt foi MODIFICADO (Alteração de: Conteúdo Tamanho Data Posse)
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=87428fc522803d31065e7bce3cf03fe475096631e5e07bbd7a0fde60c4cf25c7
  Informação atual: USER=root, GROUP=adrianamoranguita, SIZE=12, PERM=664, SHA256=2a7384e2d430093d1099cec026e31336d13b32a043851c06e606b74e7b89446e
projeto/dados.txt foi APAGADO
  Informação inicial: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=0263829989b6fd954f72baaf2fc64bc2e2f01d692d4de72986ea808f6e99813f
  Informação atual: [Ficheiro não encontrado no sistema]
projeto/novo.txt é NOVO
  Informação atual: USER=adrianamoranguita, GROUP=adrianamoranguita, SIZE=2, PERM=664, SHA256=a4fb621495a0122493b2203591c448903c472e306a1ede54fabad829e01075c0
Resumo da Verificação: 3 verificados | 1 Modificados | 1 Apagados | 1 Novos
```

5.31 Tratamento de nomes com espaços: ./vp.sh -p 'com espacos' -f bd_sp

```
Base de dados de proteção bd_sp foi criada com sucesso.
Ficheiros processados: 1
Diretórios protegidos: com espacos
```

5.32 Base de dados dentro do diretório protegido: ./vp.sh -p projeto -f projeto/bd2

```
Base de dados de proteção projeto/bd2 foi criada com sucesso.
Ficheiros processados: 3
Diretórios protegidos: projeto
```

5.33 Utilização da BD por defeito: ./vp.sh -p projeto

```
Base de dados de proteção /tmp/.pdb foi criada com sucesso.
Ficheiros processados: 4
Diretórios protegidos: projeto
```

5.34 Verificação da sintaxe: bash -n vp.sh

```
Sintaxe correta!
```

5.35 Verificação da versão do Bash: bash --version | head -1

```
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
```

6. Limitações

1. Só ficheiros regulares. A recolha usa find -type f: diretórios vazios, permissões de diretórios e ligações simbólicas não são monitorizados, pelo que criar ou apagar um diretório vazio não é detetado.
2. Diretórios sobrepostos. Com -p /a /a/b, os ficheiros de /a/b entram duas vezes na BD e uma alteração é reportada e contada duas vezes (teste 22). Uma correção seria ignorar, em gerar_base_dados, os caminhos já escritos.
3. Caminhos relativos. A BD guarda os caminhos como foram escritos; verificar a partir de outro diretório faz aparecer tudo como APAGADO (teste 24). Devem usar-se caminhos absolutos.
4. Diretório que começa por -. O -- resolve a leitura das opções, mas o find continua a ler -nome como expressão: o vp termina com sucesso e uma BD vazia (teste 23). Funciona com ./-nome ou caminho absoluto.
5. Nomes com |. O | é o separador de campos, por isso um ficheiro com | no nome corrompe o registo (teste 25). Nomes com mudança de linha também não são suportados na leitura da BD.
6. Ficheiros ilegíveis. Sem permissão de leitura, o SHA-256 fica vazio na BD e uma alteração de conteúdo só se vê pelo tamanho ou pela data (teste 26).
7. A BD não está blindada. O chmod 444 só trava a escrita acidental ou por utilizadores sem privilégios: o dono ou o root podem apagá-la ou substituí-la (é o que o próprio rm -f do vp faz), e a BD não tem assinatura nem hash próprio. Quem altera um ficheiro e a BD ao mesmo tempo não é detetado; o ideal seria guardar a BD num suporte só de leitura.
8. Código de saída. O vp -v termina com 0 mesmo quando encontra alterações (testes 9 a 11), pelo que não se presta, tal como está, a ser usado por outros scripts ou pelo cron.
9. Um só padrão -e. Várias ocorrências de -e não se acumulam (vale a última), e o padrão é comparado com o caminho completo, não só com o nome.
10. Desempenho. Cada ficheiro implica vários processos (stat, sha256sum, awk) e o SHA-256 lê todo o conteúdo, pelo que a verificação de muitos ficheiros ou de ficheiros grandes é lenta.

7. Conclusão

O vp cumpre os objetivos do trabalho: a Proteção cria uma BD com os metadados e o SHA-256 dos ficheiros dos diretórios indicados, e a Verificação deteta ficheiros novos, apagados e alterações de conteúdo, tamanho, permissões, data e posse, com resumo final e alertas opcionais para o logger. Os testes confirmaram cada deteção isoladamente e em conjunto, o comportamento do -e, a validação de erros e a proteção da BD anterior em caso de falha.

O desenvolvimento deu-nos três lições práticas: o getopts não serve quando as opções podem vir depois dos argumentos; arrays são mais seguros do que strings para listas de nomes; e um array associativo substitui com vantagem uma pesquisa com grep por ficheiro. Os testes também revelaram limitações que não esperávamos (diretórios sobrepostos, caminhos relativos), o que mostra que testar só o caso normal não chega.

Como trabalho futuro, as melhorias mais naturais são eliminar duplicados na BD, converter os caminhos para absolutos, fazer o vp -v devolver um código de saída diferente de zero quando há alterações, e proteger a própria BD com um hash guardado fora do sistema monitorizado.

8. Bibliografia

GNU Project. (2023). *Bash Reference Manual* (Version 5.2). Free Software Foundation. Disponível em: https://www.gnu.org/software/bash/manual/